

```

In [1]: import numpy as np

WIDTH, HEIGHT = 200, 300
SUGAR = (175, 100) # sugar position
TRIALS = 1000

# --- Part (a): Simple Random Walk ---
def random_walk():
    x, y = 0, 0
    steps = 0
    while (x, y) != SUGAR:
        d = np.random.random()
        if d <= 0.25: # North
            y += 1
        elif d <= 0.5: # East
            x += 1
        elif d <= 0.75: # South
            y -= 1
        else: # West
            x -= 1
        # boundaries (reflecting)
        x = min(max(x, 0), WIDTH)
        y = min(max(y, 0), HEIGHT)
        steps += 1
    return steps

# Run many trials and average
steps_list = [random_walk() for _ in range(100)]
print("Part (a): Average steps =", np.mean(steps_list))

```

Part (a): Average steps = 281387.97

```

In [4]: def self_avoiding_walk():
    x, y = 0, 0
    visited = set([(x, y)])
    steps = 0

    while True:
        # If sugar found
        if (x, y) == SUGAR:
            return True, steps

        # Find unvisited neighbors
        neighbors = []
        if x+1 <= WIDTH and (x+1, y) not in visited:
            neighbors.append((x+1, y))
        if x-1 >= 0 and (x-1, y) not in visited:
            neighbors.append((x-1, y))
        if y+1 <= HEIGHT and (x, y+1) not in visited:
            neighbors.append((x, y+1))
        if y-1 >= 0 and (x, y-1) not in visited:
            neighbors.append((x, y-1))

        # No available moves → trapped
        if not neighbors:

```

```

        return False, steps

        # Choose a random unvisited neighbor
        x, y = neighbors[np.random.randint(len(neighbors))]
        visited.add((x, y))
        steps += 1

# Run 1000 trials
successes = 0
steps_success = []
for _ in range(TRIALS):
    success, steps = self_avoiding_walk()
    if success:
        successes += 1
        steps_success.append(steps)

print("Part (b): Success probability =", successes / TRIALS)
if steps_success:
    print("Average steps for successful runs =", np.mean(steps_success))

```

Part (b): Success probability = 0.0

```

In [7]: import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from random import uniform

%matplotlib inline

WIDTH, HEIGHT = 200, 300
SUGAR = (175, 100)
x0, y0 = 50, 50

def move_particle(x, y):
    d = uniform(0, 1)
    dx, dy = 0, 0

    if d <= 0.25:
        dy = 1 # North
    elif d <= 0.5:
        dx = 1 # East
    elif d <= 0.75:
        dy = -1 # South
    else:
        dx = -1 # West

    new_x = min(max(x + dx, 0), WIDTH)
    new_y = min(max(y + dy, 0), HEIGHT)
    return new_x - x, new_y - y

def get_pos():
    x, y = x0, y0
    max_steps = 10000
    step = 0
    while (x, y) != SUGAR and step < max_steps:
        dx, dy = move_particle(x, y)

```

```

        x += dx
        y += dy
        step += 1
        yield x, y
    yield x, y

def init():
    ax.set_xlim(0, WIDTH)
    ax.set_ylim(0, HEIGHT)
    line.set_data([], [])
    dot.set_data([x0], [y0]) # Ensure sequence
    height_text.set_text(f'(x,y) = ({x0},{y0})')
    return line, dot, height_text

def animate(pos):
    x, y = pos
    xdata.append(x)
    ydata.append(y)

    if len(xdata) > 1000:
        xdata.pop(0)
        ydata.pop(0)

    line.set_data(xdata, ydata)
    dot.set_data([x], [y])
    height_text.set_text(f'(x,y) = ({x},{y})')

    current_xlim = ax.get_xlim()
    current_ylim = ax.get_ylim()

    if x < current_xlim[0] or x > current_xlim[1]:
        ax.set_xlim(min(x-10, current_xlim[0]), max(x+10, current_xlim[1]))
    if y < current_ylim[0] or y > current_ylim[1]:
        ax.set_ylim(min(y-10, current_ylim[0]), max(y+10, current_ylim[1]))

    return line, dot, height_text

fig, ax = plt.subplots(figsize=(8, 6))
ax.set_xlim(0, WIDTH)
ax.set_ylim(0, HEIGHT)
ax.set_aspect('equal')

ax.plot(SUGAR[0], SUGAR[1], 'r*', markersize=12, label="Sugar")

line, = ax.plot([], [], lw=1, alpha=0.7, label="Path")
dot, = ax.plot([x0], [y0], 'bo', markersize=5, label="Ant")
xdata, ydata = [], []
height_text = ax.text(0.02, 0.95, f'(x,y) = ({x0},{y0})', transform=ax.transAxes,
                      bbox=dict(boxstyle="round,pad=0.3", facecolor="white", alpha=0

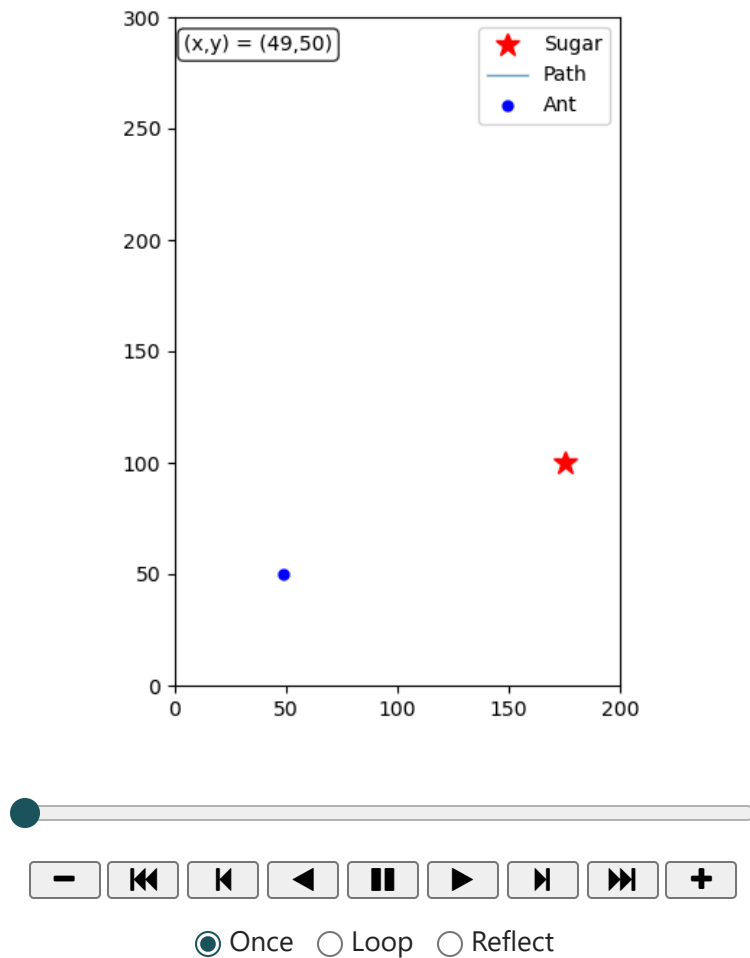
interval = 100
ani = animation.FuncAnimation(fig, animate, get_pos, init_func=init,
                              interval=interval, repeat=False, blit=False,
                              save_count=1000, cache_frame_data=False)

```

```
plt.legend()
```

animation size has reached 20975361 bytes, exceeding the limit of 20971520.0. If you're sure you want a larger animation embedded, set the `animation.embed_limit` rc parameter to a larger value (in MB). This and further frames will be dropped.

Out[7]:



In []: